ws-CIR 1.0

Web Service Common Interoperability Registry

OpenO&M Candidate Standard 19 June 2015

This Document

http://www.openoandm.org/ws-cir/1.0/ws-cir.html

WSDL

http://www.openoandm.org/ws-cir/1.0/wsdl/CommonInteroperabilityRegistry.wsdl

Packaged Specification

http://www.openoandm.org/ws-cir/ws-cir-1.0.zip

Latest Version

http://www.openoandm.org/ws-cir

Editors

MIMOSA

Avin Mathew, Assetricity

Ken Bever, Assetricity

ISA

Dennis Brandl, BR&L Consulting

Abstract

This document defines the OpenO&M Web Service Common Interoperability Registry (ws-CIR). The ws-CIR specification is a standards-based, vendor-neutral approach for the construction of an object registration server. The specification defines an underlying logical data model, the services for the registry, and a normative XML Schema/WSDL specification as well as a message model for the services.

Status

Implementers should be aware that this specification is not stable. *Implementers who are not taking part in the discussions are likely to find the specification changing out from under them in incompatible ways.* Vendors interested in implementing this specification before it eventually reaches the final Standard stage should join the MIMOSA mailing lists and take part in the discussions.

The latest stable version of the editor's draft of this specification is always available on the MIMOSA ws-CIR Git repository (https://github.com/mimosa-org/ws-cir).

If you wish to make comments regarding this specification in a manner that is tracked by OpenO&M, please submit them via the public bug database (https://github.com/mimosa-org/ws-cir/issues). You can alternatively contact MIMOSA directly (http://www.mimosa.org/contact) and arrangements will be made to transpose appropriate remarks to the public bug database. All feedback is welcome.

Notices

All capitalized terms in the following text have the meanings assigned to them in the MIMOSA Intellectual Property Rights Policy (the "MIMOSA IPR Policy"). The full Policy may be found at the MIMOSA website (http://www.mimosa.org/sites/default/files/policies-charters/MIMOSA_IPR_Policy.pdf).

The limited permissions granted above are perpetual and will not be revoked by MIMOSA or its successors or assigns.

This document and the information contained herein is provided on an "AS IS" basis and MIMOSA DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY OWNERSHIP RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

MIMOSA requests that any MIMOSA Party or any other party that believes it has patent claims that would necessarily be infringed by implementations of this MIMOSA Final Deliverable, to notify MIMOSA TC Administrator and provide an indication of its willingness to grant patent licenses to such patent claims in a manner consistent with the IPR Mode of the MIMOSA Technical Committee that produced this deliverable.

MIMOSA invites any party to contact the MIMOSA TC Administrator if it is aware of a claim of ownership of any patent claims that would necessarily be infringed by implementations of this MIMOSA Final Deliverable by a patent holder that is not willing to provide a license to such patent claims in a manner consistent with the IPR Mode of the MIMOSA Technical Committee that produced this MIMOSA Final Deliverable. MIMOSA may include such claims on its website, but disclaims any obligation to do so.

MIMOSA takes no position regarding the validity or scope of any intellectual property or other rights that might be claimed to pertain to the implementation or use of the technology described in this MIMOSA Final Deliverable or the extent to which any license under such rights might or might not be available; neither does it represent that it has made any effort to identify any such rights. Information on MIMOSA's procedures with respect to rights in any document or deliverable produced by a MIMOSA Technical Committee can be found on the MIMOSA website. Copies of claims of rights made available for publication and any assurances of licenses to be made available, or the result of an attempt made to obtain a general license or permission for the use of such proprietary rights by implementers or users of this MIMOSA Final Deliverable, can be obtained from the MIMOSA TC Administrator. MIMOSA makes no representation that any information or list of intellectual property rights will at any time be complete, or that any claims in such list are, in fact, Essential Claims.

The OpenO&M ws-CIR is released under the MIMOSA License Agreement (http://www.mimosa.org/sites/default/files/policies-charters/MIMOSA_License_Agreement.pdf).

Table of Contents

1 Introduction

The Web Service Common Interoperability Registry (ws-CIR) Specification provides a normative specification for the implementation of an object registry for operations and maintenance. It consists of:

- A functional specification (this document)
- WSDL service definitions (with associated XML Schema definitions)
- Message model based on Open Applications Group Integration Specification (OAGIS)

The specification should be sufficiently detailed so that an implementation of the ws-CIR server can be developed unambiguously.

1.2 Notational Conventions

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119 (http://www.ietf.org/rfc/rfc2119.txt).

This specification uses the following syntax to define XML structures: `Element Name (Type) [Cardinality]`. The namespaces for Types are defined in the following section. For example, the Topic element defined as an XML Schema `string` with one to many cardinality would be defined as: `Topic (xs:string) [1..*]`.

1.3 Namespaces

The following namespaces are used in this document:

Prefix	Namespace
xs	http://www.w3.org/2001/XMLSchema
cct	urn:un:unece:uncefact:documentation:standard:CoreComponentType:2
cir	http://www.openoandm.org/ws-cir/

2 Logical Data Model

This section presents the data model used within the ws-CIR specification as part of its conceptual design. A ws-CIR server implementation MAY use this data model as a physical data model for data persistence.

2.1 Primitive Data Types

2.1.1 XML Schema Types

As the ws-CIR Services use XML Schema for schema definitions used by the WSDL services and message model, all primitive types used in the ws-CIR model are derived from XML Schema primitive types.

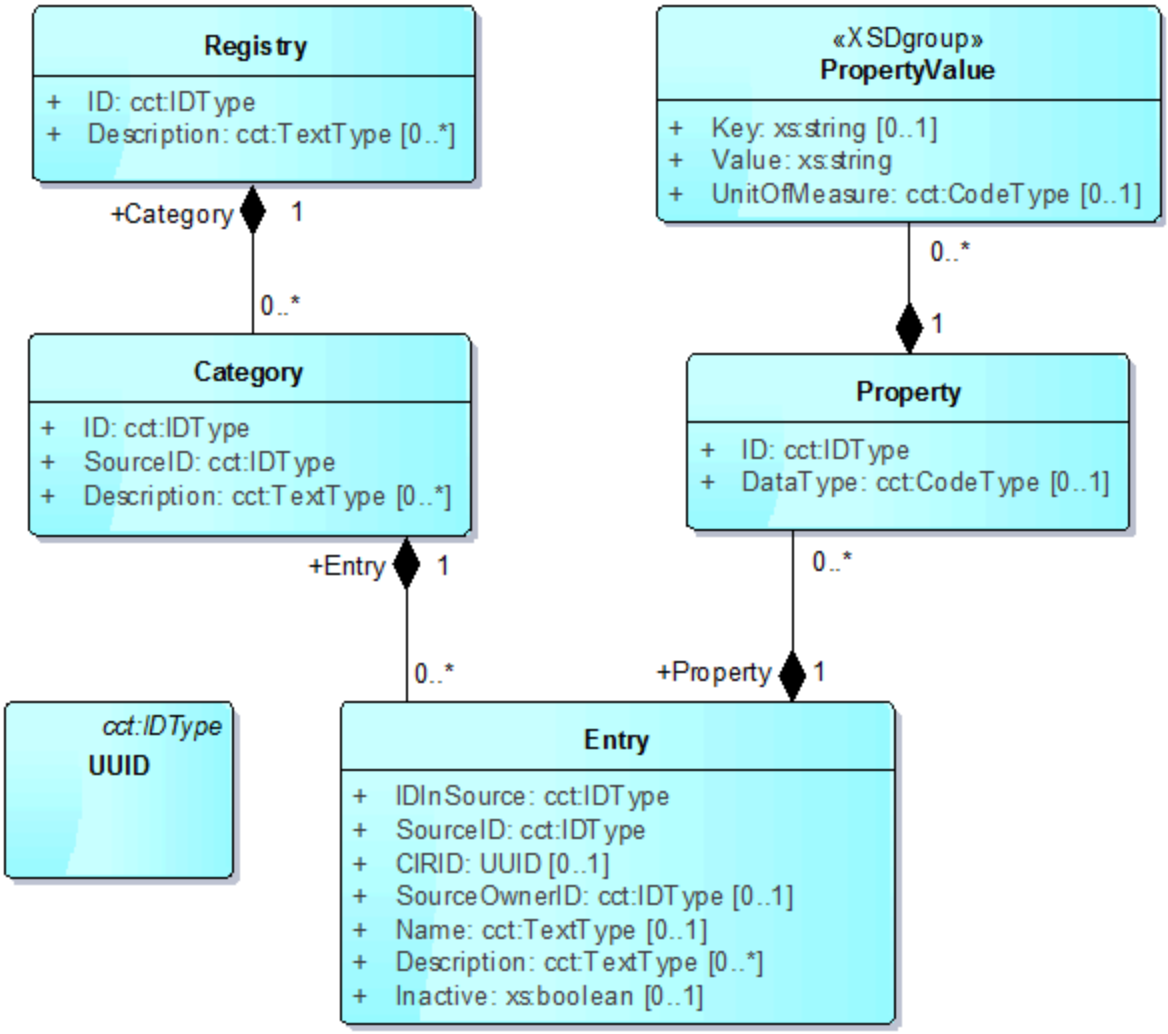
The namespace prefix `xs` is used to denote the XML Schema types in any UML diagrams.

2.1.2 Core Component Types

The UN/CEFACT Core Component Types 2.01 (http://www.unece.org/cefact/codesfortrade/ccts_datatypecatalogue.html), which derive from XML Schema primitive types and define basic data element types for semantic interoperability, are used in place of most primitive types in the data model. For most attributes, the usage of the Core Component Types is not explicitly addressed by this ws-CIR specification, and their inclusion is for future versions of the ws-CIR specification. However, there are two locations within the data model that immediately necessitate their inclusion: (1) the use of the language/locale attribute on TextType for Registry/Category/Entry Description attributes, and (2) the code list metadata on CodeType for the Property UnitOfMeasure attribute.

The namespace prefix `cct` is used to denote the Core Component Types in any UML diagrams.

2.2 UML Model



2.3 Registry

A Registry is the container object for a set of Categories. Examples of multiple registries include: test registry, active registry, local site registry, global corporate registry.

Attribute	Description	Cardinality
ID	User defined identifier of the registry. This MUST be unique within the ws-CIR server. For example: • Registration Server A • Test Registry • Finance System Registry A value based on ISO/IEC 9834-8 UUID MAY be used to ensure global uniqueness.	1

Attribute	Description	Cardinality
Description	Description and expected use of the registry. Multiple values are allowed for multiple languages or alternate descriptions. The language/locale is specified through the UN/CEFACT TextType metadata attributes.	0..*

Primary Key: Registry.ID

2.4 Category

A Category object is the container object for a set of Entries. Categories define sets of related or potentially related Entries. For example, a Category may be defined for equipment hierarchy level names (Enterprise, Site, Area, Work Center, Work Unit), which have alternate names on different systems. The combination of ID and SourceID MUST be unique within a Registry.

Attribute	Description	Cardinality
ID	User defined identifier of the category. For example: • Asset • Asset_Type • Segment • Segment_Type • Meas_Location • Meas_Loc_Type • Network • Network_Type • P&ID Diagram Object • Batch Status • Production Status • Equipment Status	1
SourceID	Identification of the category. May define the organization and specification name for the category. For example: • MIMOSA OSA-EAI V3 • ISA 88 BatchStatus • ISA 95-2000 EquipmentModel • B2MML.EquipmentModel • ISA88.RecipeModel1995 • BatchML.RecipeModelV4.04.01 • ChemCompany.RefineryModelV2.1 • ShippingCompany.TransportCode	1
Description	Description and expected use of the category. Multiple values are allowed for multiple languages or alternate descriptions. The language/locale is specified through the UN/CEFACT TextType metadata attributes.	0..*

Primary Key: Registry.ID, Category.ID, Category.SourceID

2.5 Entry

Entries define named element and properties with an identifier local to the owning application and a possible global ID (CIRID) that defined equivalent Entries in other applications. For example, the record TC101 in System A may be the equivalent of record UNIT101.TOP_TEMP in System B. The combination of IDinSource and SourceID MUST be unique within a Category.

Attribute	Description	Cardinality
IDInSource	User defined identification of the entry in the source system. This may be the primary key within the source system or another unique value that can be used to distinguish objects within the source system.	1
SourceID	Identification of the source system. For example: • Engineering DB #234 • Supplier A MIMOSA CRIS Registry Database #1 • EAM/CMMS System B	1
CIRID	System-assigned universally unique ID for the entry based on ISO/IEC 9834-8 UUID. Used to correlate multiple entries to identify logical equivalent entries (i.e. multiple entries with the same CIRID are equivalent objects).	0..1
SourceOwnerID	Organization that has responsibility for the source system or entity namespace. For example: • Oil Company A • Chem Company B • System Supplier A • System Supplier B	0..1
Name	Name of the entry. This is the source system's external identification of the entry. It is often a label the user will see on a screen for this object in the source system. Usually locally unique for the user, but may not be the source system's internal primary key/unique identifier.	0..1
Description	Description of the entry. Multiple values are allowed for multiple languages or alternate descriptions. The language/locale is specified through the UN/CEFACT TextType metadata attributes.	0..1
Inactive	Boolean flag where FALSE or absent indicates the entry is active and available for use while TRUE indicates the entry is inactive. Examples of inactive entries may be data that is entered but the source system is not yet available or in use.	0..1

Primary Key: Registry.ID, Category.ID, Category.SourceID, Entry.IDInSource, Entry.SourceID

Entries with the same CIRID are considered equivalent objects. For example, for the following set of Entries (not all columns are shown), the first three Entries are equivalent.

IDInSource	SourceID	CIRID	Name
234443	System A	550e8400-e29b-41d4-a716-446655440000	Loop 106
423ABC	System B	550e8400-e29b-41d4-a716-446655440000	Cmn Loop 106
TIC-106	System C	550e8400-e29b-41d4-a716-446655440000	Top Temp Control
TIC-8106	System C	550e8400-e29b-41d4-a716-446655448778	Top Temp Control

2.6 Property

A Property defines an attribute or characteristic of an Entry. Properties may be used to help identify equivalent Entries. The Properties should be a small set of attributes that may be needed to link systems together, and are not intended to be a global property master registry.

Attribute	Description	Cardinality
ID	User defined identification of the property. This MUST be unique for a registry entry.	1
PropertyValue	Value of the property. See PropertyValue	0..*
DataType	Data type of the value.	0..1

Primary Key: Registry.ID, Category.ID, Category.SourceID, Entry.IDInSource, Entry.SourceID, Property.ID

2.7 PropertyValue

A PropertyValue is a group of attributes that form the value of an Entry Property. The PropertyValue can be a key-value pair or a single value with or without a unit of measure.

Attribute	Description	Cardinality
Key	String-serialized key for the key-value pair of the property. Is not required if the value is not a key-value pair.	0..1
Value	String-serialized value of the property	1
UnitOfMeasure	Unit of measure of the value. The code list MAY be specified through the UN/CEFACT CodeType list metadata attributes.	0..1

A JSON representation MAY be used to represent the Value, similar to the OpenO&M Defined Properties ().

3 Service Definitions

This section defines the detailed format for the ws-CIR Service definitions.

3.1 ws-CIR Command Services

The Command Services exposed by the ws-CIR allow a client to create/update/delete registry data. All operations MUST be atomic, and there MUST NOT be no partial creates/updates/deletions if a fault is thrown during the invocation of a service.

3.1.1 Create Registry

Name	CreateRegistry
Description	Creates a new Registry, new Category in a Registry, new Entries in a Category, and Properties with Values in an Entry.
Input	Registry (cir:Registry) [1..*] The Registry/Category/Entry/Property structure to create within the ws-CIR server. CreateCIRID (xs:boolean (http://www.w3.org/TR/xmlschema-2/#boolean)) [1] This flag indicates whether CIRIDs are created and allocated to the supplied Entries.
Behavior	If the ws-CIR server is configured to not allow new Registry objects and a new RegistryID is supplied, then the ws-CIR server MUST throw a CreateRegistryFault. If the ws-CIR server is configured to not allow new Category objects and a new CategoryID and SourceID are supplied, then the ws-CIR server MUST throw a CreateCategoryFault. If there is a duplicate Entry (same primary key as an existing Entry), then the ws-CIR server MUST throw a DuplicateEntryFault. If there is a duplicate Property (same primary key as an existing Property), then the ws-CIR server MUST throw a DuplicatePropertyFault. If CreateCIRID is TRUE, then the ws-CIR server MUST generate new UUIDs for any Entries supplied without a CIRID. If CreateCIRID is FALSE, then the CIRID field is not supplemented with UUIDs.
Output	N/A
Faults	CreateRegistryFault CreateCategoryFault DuplicateEntryFault DuplicatePropertyFault

3.1.2 Create Equivalent Entries

Name	CreateEquivalentEntries

Description	Creates Entries and associated Properties, and links a new Entry to an existing equivalent Entry.
Input	EquivalentEntry (cir:EquivalentEntry) [1..*], composed of: ExistingIDInSource (cct:IDType) [1] ExistingSourceID (cct:IDType) [1] RegistryID (cct:IDType) [1] CategoryID (cct:IDType) [1] CategorySourceID (cct:IDType) [1] Entry (cir:Entry) [1] The Entry/Property structure to create within the ws-CIR server.
Behavior	If the Entry identified by the ExistingIDInSource and ExistingSourceID is not found, then the ws-CIR server MUST throw an EntryNotFoundFault. If the Registry identified by the RegistryID is not found, then the ws-CIR server MUST throw a RegistryNotFoundFault. If the Category identified by the CategoryID and CategorySourceID is not found, then the ws-CIR server MUST throw a CategoryNotFoundFault. If there is a duplicate Entry (same primary key as an existing Entry), then the ws-CIR server MUST throw a DuplicateEntryFault. If only the existing Entry or both the existing Entry and supplied Entry have a CIRID, then the ws-CIR server MUST apply the existing Entry CIRID to both Entries. If the existing Entry does not have a CIRID but the supplied Entry does, the ws-CIR server MUST apply the supplied Entry CIRID to both Entries. If neither Entry has a CIRID specified, then the ws-CIR server MUST create and apply new CIRID to both Entries.
Output	N/A
Faults	EntryNotFoundFault RegistryNotFoundFault CategoryNotFoundFault DuplicateEntryFault

3.1.3 Update Registry

Name	UpdateRegistry
Description	Updates the attributes of existing Registries, Categories, Entries or Properties.
Input	Registry (cir:Registry) [1..*]

Behavior	All attributes of a Registry/Category/Entry/Property object apart from its primary key are updated based on the supplied data. If a Registry identified by the ID is not found, then the ws-CIR server MUST throw a RegistryNotFoundFault. If a Category identified by the ID and SourceID is not found, then the ws-CIR server MUST throw a CategoryNotFoundFault. If an Entry identified by the IDInSource and SourceID is not found, then the ws-CIR server MUST throw an EntryNotFoundFault. If a Property identified by the ID is not found, then the ws-CIR server MUST throw a PropertyNotFoundFault.
Output	N/A
Faults	RegistryNotFoundFault CategoryNotFoundFault EntryNotFoundFault PropertyNotFoundFault

3.1.4 Update Entry CIRID

Name	UpdateEntryCIRID
Description	Replaces the CIRID field on matching Entries with a new CIRID value.
Input	OldCIRID (cir:UUID) [1..*] NewCIRID (cir:UUID) [1]
Output	N/A
Faults	N/A

3.1.5 Delete Registry

Name	DeleteRegistry
Description	Deletes the specified Registry along with its Categories, Entries and Properties.
Input	RegistryID (cct:IDType) [1]
Behavior	If the Registry identified by the RegistryID is not found, then the ws-CIR server MUST throw a RegistryNotFoundFault.
Output	N/A
Faults	RegistryNotFoundFault

3.1.6 Delete Category

Name	DeleteCategory
Description	Deletes the specified Category along with its Entries and Properties.
Input	RegistryID (cct:IDType) [1] CategoryID (cct:IDType) [1] CategorySourceID (cct:IDType) [1]
Behavior	If the Registry identified by the RegistryID is not found, then the ws-CIR server MUST throw a RegistryNotFoundFault. If the Category identified by the CategoryID and CategorySourceID is not found, then the ws-CIR server MUST throw a CategoryNotFoundFault.
Output	N/A
Faults	RegistryNotFoundFault CategoryNotFoundFault

3.1.7 Delete Entries

Name	DeleteEntries
Description	Deletes the specified Entries along with its Properties.
Input	EntryIdentifier (cir:EntryIdentifier) [1..*], composed of: RegistryID (cct:IDType) [1] CategoryID (cct:IDType) [1] CategorySourceID (cct:IDType) [1] EntryIDInSource (cct:IDType) [1] EntrySourceID (cct:IDType) [1]
Behavior	If a Registry identified by the ID is not found, then the ws-CIR server MUST throw a RegistryNotFoundFault. If a Category identified by the ID and SourceID is not found, then the ws-CIR server MUST throw a CategoryNotFoundFault. If an Entry identified by the IDInSource and SourceID is not found, then the ws-CIR server MUST throw an EntryNotFoundFault.
Output	N/A
Faults	RegistryNotFoundFault CategoryNotFoundFault EntryNotFoundFault

3.1.7 Delete Properties

Name	DeleteProperties
Description	Deletes the specified Properties.
Input	PropertyIdentifier (cir:PropertyIdentifier) [1..*], composed of: RegistryID (cct:IDType) [1] CategoryID (cct:IDType) [1] CategorySourceID (cct:IDType) [1] EntryIDInSource (cct:IDType) [1] EntrySourceID (cct:IDType) [1] PropertyID (cct:IDType) [1]
Behavior	If a Registry identified by the ID is not found, then the ws-CIR server MUST throw a RegistryNotFoundFault. If a Category identified by the ID and SourceID is not found, then the ws-CIR server MUST throw a CategoryNotFoundFault. If an Entry identified by the IDInSource and SourceID is not found, then the ws-CIR server MUST throw an EntryNotFoundFault. If a Property identified by the ID is not found, then the ws-CIR server MUST throw a PropertyNotFoundFault.
Output	N/A
Faults	RegistryNotFoundFault CategoryNotFoundFault EntryNotFoundFault PropertyNotFoundFault

3.2 ws-CIR Query Services

The Query Services exposed by the ws-CIR allow a client to retrieve registry data. The client MAY use the Wildcard Specification in designated fields for advanced string matching.

3.2.1 Get Registry

Name	GetRegistry
Description	Returns all Registries, Categories, Entries and Properties filtered by the specified conditions.
Input	Registry (cir:Registry) [1..*] The Registry/Category/Entry/Property structure to create within the ws-CIR server. Filter (cir:Filter) [0..*], composed of: RegistryFilter (cir:RegistryFilter) [0..1] CategoryFilter (cir:CategoryFilter) [0..1] EntryFilter (cir:EntryFilter) [0..1] PropertyFilter (cir:PropertyFilter) [0..1]

Behavior	Each filter type within a Filter (i.e. RegistryFilter, CategoryFilter, EntryFilter, PropertyFilter) acts as a logical AND filter. For example, if the Registry ID value is "Test", Category ID is "Asset", and Property ID "Length" then only Entries (and associated Registry, Category and Properties) of the "Asset" Category in the "Test" Registry that have a Property of "Length" are returned. The absence of an input parameter type indicates that the data is not filtered by this facet (i.e. logical TRUE) and that all data elements are valid. Multiple filters of the same filter type are supported and act as a logical OR filter. For example, if the EntryFilter 1 Name is "P101" and the EntryFilter 2 Name is "P102", then the Entries with a Name of "P101" or "P102" are returned. Wildcards are supported on all fields within each filter type.
Output	Registry (cir:Registry) [0..*]

3.2.2 Get Equivalent Entries

Name	GetEquivalentEntries
Description	Returns any equivalent Entries to the specified Entries (i.e. by identifying all Entries with the same CIRID to the specified Entries). Multiple entries are specified by IDInSource and SourceID pairs. A TargetSourceID or list of TargetSourceIDs can be specified to filter equivalent Entries.
Input	EntryIdentifier (cir:EntryIdentifier) [1..*], composed of: RegistryID (cct:IDType) [1] CategoryID (cct:IDType) [1] CategorySourceID (cct:IDType) [1] EntryIDInSource (cct:IDType) [1] EntrySourceID (cct:IDType) [1] TargetSourceID (cct:IDType) [0..*]
Behavior	Both the specified Entry and equivalent Entries are returned to allow correlation (via CIRID) by the client. If the specified Entry does not exist or does not have a corresponding CIRID, no equivalent Entries will be returned. Nevertheless, the specified Entry is still returned with a populated CIRID field. The TargetSourceID filter only filters the equivalent Entries to those with the same SourceID. If no TargetSourceID is specified, all Entries with the same CIRID are returned. Multiple TargetSourceIDs are supported and act as a logical OR filter. Wildcards are only supported on the TargetSourceID field.
Output	Registry (cir:Registry) [0..*]

3.2.3 Get Entries By CIRID

Name	GetEntriesByCIRID

Description	Returns any Entries with the specified CIRID. An Entry is specified by CIRID. A TargetSourceID or list of TargetSourceIDs can be specified to filter returned Entries.
Input	ExistingCIRID (cir:UUID) [1] TargetSourceID (cct:IDType) [0..*]
Behavior	If there is no existing Entry with the specified CIRID, nothing is returned. Only other Entries are returned – the existing Entry is not returned as part of the result set. If no TargetSourceID is specified, all Entries with the same CIRID are returned. Multiple TargetSourceIDs are supported and act as a logical OR filter. Wildcards are only supported on the TargetSourceID field.
Output	Registry (cir:Registry) [0..*]

4 Wildcard Specification

To expand the RegistryFilter, CategoryFilter, EntryFilter and PropertyFilter beyond basic string matching, a limited subset of POSIX Regular Expression (http://pubs.opengroup.org/onlinepubs/9699919799/basedefs/V1_chap09.html) metacharacters MAY be used for more advanced string matching. The following metacharacters are supported by this specification:

. Matches any single character except for newline characters
* Matches the preceding element zero or more times
+ Matches the preceding element one or more times
? Matches the preceding element zero or one time
\ Escape character to interpret the following character as a literal character

Note The entire regular expression is implicitly anchored and its start and end. To accept all elements with the expression in the middle of their contents, use an expression similar to `.*term.*`.

5 Conformance

Any assessment of conformance of a ws-CIR implementation MUST be qualified by the following:

1. Support for Command Services
2. Support for Query Services
3. Support for Wildcard Specification
4. Support for SOAP 1.1 and SOAP 1.2 services
5. Support for specific BODs (see OAGIS-Based Message Model)
6. A statement of the total conformance concerning services supported or, in case of partial conformance, a statement identifying explicitly the areas of non-conformance

Appendix A OpenO&M Defined Properties

There are predefined OpenO&M properties that SHOULD be used to identify commonly understood relationships between entities.

ParentEntityID

The ParentEntityID contains the set of IDInSource IDs for parent object of the entity in the source's hierarchy. Multiple parent objects are specified by multiple PropertyValues.

ChildEntityID

The ChildEntityID contains the set of IDInSource IDs for child objects of the entity in the source's hierarchy. Multiple child objects are specified by multiple PropertyValues.

PossibleEquivalentEntryID

The PossibleEquivalentEntryID contains a set of target entities which are possibly equivalent to the entity. This allows for automated equivalency determination. Each returned target entry contains the following set of information:

- IDInSource
- SourceID
- PercentLikelihood [Optional]

The property value should follow the JSON format for the array of objects of equivalent IDs. For example:

```
[ { "IDInSource": "TIC101", "SourceID": "Thor" } ]
```

```
[ { "IDInSource": "TIC101", "SourceID": "Thor", "PercentLikelihood": 20 } ]
```

```
[
  { "IDInSource": "TIC101", "SourceID" : "Thor", "PercentLikelihood": 20 },
  { "IDInSource": "T101", "SourceID" : "Apollo" }
]
```

Annex A OAGIS-Based Message Model

A set of OAGIS-based Business Object Documents (BODs) are defined in this specification for use in a messaging-based environment (e.g. with the OpenO&M ws-ISBM (http://www.openoandm.org/ws-isbm)). The messages (located in the `bod` directory of the package) are defined as XML Schema and reference the OAGIS Platform Specification 1.2.1 (located in the `bod/oagis` directory of the package) as well as the ws-CIR Service Definition XML Schema (located in the `xsd` directory of the package). While the included OAGIS XML Schemas have been edited to serve the purposes of this ws-CIR specification, they remain a proper subset of OAGIS and are also compatible with the original full definition.

BOD Catalogue

The following table is a complete listing of the ws-CIR BODs with the corresponding Verb and Noun.

BOD	Verb	Noun
AcknowledgeEquivalentEntries (bod/AcknowledgeEquivalentEntries.xsd)	Acknowledge	CreateEquivalentEntries faults
AcknowledgeRegistry (bod/AcknowledgeRegistry.xsd)	Acknowledge	CreateRegistry faults
CancelCategory (bod/CancelCategory.xsd)	Cancel	DeleteCategory
CancelEntries (bod/CancelEntries.xsd)	Cancel	DeleteEntries
CancelProperties (bod/CancelProperties.xsd)	Cancel	DeleteProperties
CancelRegistry (bod/CancelRegistry.xsd)	Cancel	DeleteRegistry
ChangeEntryCIRID (bod/ChangeEntryCIRID.xsd)	Change	UpdateEntryCIRID
ChangeRegistry (bod/ChangeRegistry.xsd)	Change	UpdateRegistry
GetEquivalentEntries (bod/GetEquivalentEntries.xsd)	Get	GetEquivalentEntries
GetEntriesByCIRID (bod/GetEntriesByCIRID.xsd)	Get	GetEntriesByCIRID
GetRegistry (bod/GetRegistry.xsd)	Get	GetRegistry
ProcessEquivalentEntries (bod/ProcessEquivalentEntries.xsd)	Process	CreateEquivalentEntries
ProcessRegistry (bod/ProcessRegistry.xsd)	Process	CreateRegistry
RespondRegistry (bod/RespondRegistry.xsd)	Respond	UpdateRegistry faults
ShowEquivalentEntries (bod/ShowEquivalentEntries.xsd)	Show	GetEquivalentEntriesResponse

BOD	Verb	Noun
ShowEntriesByCIRID (bod/ShowEntriesByCIRID.xsd)	Show	GetEntriesByCIRIDResponse
ShowRegistry (bod/ShowRegistry.xsd)	Show	GetRegistryResponse

Request/Response BODs

The following table correlates the ws-CIR response BOD for a given ws-CIR request BOD.

Request BOD	Response BOD
ProcessRegistry (bod/ProcessRegistry.xsd)	AcknowledgeRegistry (bod/AcknowledgeRegistry.xsd)
ProcessEquivalentEntries (bod/ProcessEquivalentEntries.xsd)	AcknowledgeEquivalentEntries (bod/AcknowledgeEquivalentEntries.xsd)
ChangeRegistry (bod/ChangeRegistry.xsd)	RespondRegistry (bod/RespondRegistry.xsd)
GetRegistry (bod/GetRegistry.xsd)	ShowRegistry (bod/ShowRegistry.xsd)
GetEquivalentEntries (bod/GetEquivalentEntries.xsd)	ShowEquivalentEntries (bod/ShowEquivalentEntries.xsd)
GetEntriesByCIRID (bod/GetEntriesByCIRID.xsd)	ShowEntriesByCIRID (bod/ShowEntriesByCIRID.xsd)

Note There is no response to ChangeEntryCIRID as the ws-CIR model returns no value nor throws any faults.

Note There are no responses for Cancel BODs for consistency with the OAGIS model.

Usage of OAGIS Elements

This section clarifies the ws-CIR usage of certain OAGIS message elements. It is assumed the reader has some familiarity with the OAGIS XML Schema structure.

BOD Attributes

The `releaseID` attribute of `BusinessObjectDocumentType` MUST be set to the version of the OAGIS release that is referenced by the BOD; the value of this attribute for ws-CIR 1.0 BODs is `1.2.1`.

The `versionID` attribute of `BusinessObjectDocumentType` MUST be set to the version of the BOD (e.g. `1.0`).

Application Area

All BOD Application Areas should at a minimum include the `Sender LogicalID`, `CreationDateTime` and `BODID`.

Verbs

All Process, Change and Cancel verbs SHOULD only use a snapshot approach (where the full entity is sent). An `ActionExpression` is OPTIONAL (and should be ignored) as the noun will be implicitly processed according to the ws-CIR service invoked (e.g. the ProcessRegistry BOD with a CreateRegistry noun will *add* a new registry).

ws-CIR server implementations MUST support all `ProcessType` `acknowledgeCode` s and `ChangeType` `responseCode` s for message confirmation and error reporting of Process and Change BOD requests/responses.

The result paging attributes of the Get and Show verbs are not supported due to the nested structure of the result set.

The `OriginalApplicationArea` from Acknowledge, Respond and Show BODs can be omitted if other message correlation functionality is used.

Error Handling

As opposed to the WSDL implementation which can only return a single fault during an operation, this message model allows for multiple faults in a response. A ws-CIR server implementation MAY return a single or multiple faults in a response.

Acknowledgements

The following individuals have participated in the creation of this specification and are gratefully acknowledged:

- Brandon Mathis, Assetricity
- Dave Emerson, Yokogawa